The Feasibility of using Livox Horizon for Vehicle Localisation and Obstacle Detection

June 2, 2020

1 Introduction

The focus in this study is on verifying if the Livox Horizon LiDAR can be utilised for the localisation of a ground vehicle in vineyards and for obstacle detection.

2 Comparison of range sensing technologies

In this study, we make a distinction between two classes of LiDARs, namely, Rotary and Solid State. Rotary LiDARs have multiple laser-receiver pairs stacked in a vertical row. For instance, the Velodyne HDL64E sensor, shown in **figure 1a**, contains 64 laser-receiver pairs. Rotating all pairs as a whole leads to a collection of parallel rings of points at different depths. On the other hand, solid state sensors come in various implementations, such as micro-electro-mechanical-system (MEMS) scanning or optical phase array (OPA). For instance, the Livox Horizon LiDAR, shown in **figure 1b**, is an opto-semiconductor device that operates like a direct time of flight system and utilises multiple light-photosensor pairs. Used in combination with a pulse modulated light source, the direct time of flight system can obtain distance information by calculating the phase information from light emission and reception timing.

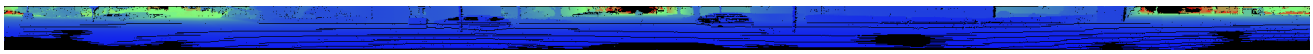
Solid state sensors generate many features at a much lower cost when compared to their rotary counterparts. But these features bring significant challenges to LiDAR data processing including:

- **Small field-of-view:** Livox Horizon has a front facing, conical shaped field-of-view spanning 81.7^o horizontally and 25.1^o vertically. The reduced field-of-view will lead to very fewer features in a frame, making the subsequent feature matching prone to degenerate and easily disturbed by moving objects. This limitation presents a challenge for localisation and mapping. Hence it is usually recommended that these types of sensors be used as range sensors for obstacle detection. However, for navigation, they are usually recommended only for slow moving vehicles (5 km/hour) and should be used in conjunction with its internal IMU.

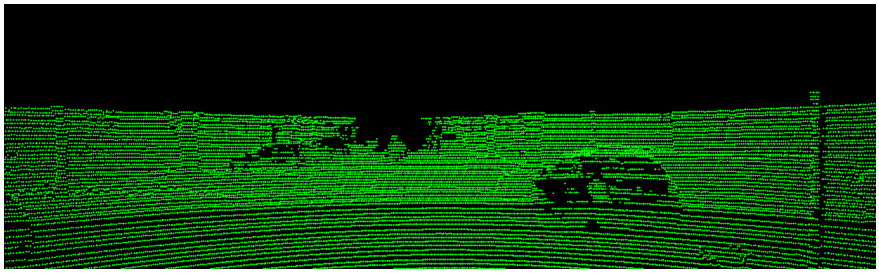
(a) Velodyne HDL64E, $75k.

(b) Livox Horizon, $800.

Figure 1: The Velodyne-HDL64E is a 64-line mechanically spinning scanner (the LiDAR head spins around 360^o), while the Livox-Horizon is a solid state scanner.



(a) The HDL64E generates 360^o scans where each scan is composed of 64 lines. This data can be encoded into a dense depth image with a size of 1947×64. Black pixels represents no laser return due to either blind spots or out of range.



(b) The scanning pattern of the velodyne HDL64E can be visualised by projecting the 3D scan into a hypothetical image plane at a given field of view. In this figure, an image plane with size 1241×376 is assumed, where the remaining data outside this field of view is cropped out. This size is chosen in order to be consistent with the pattern image size generated from the Livox Horizon scanner.

Figure 2: Sample depth image and scanning pattern of the HDL64E scanner.

- **Irregular scanning pattern:** The regular scanning pattern of the HDL64E, shown in **figure 2b** greatly simplifies 3D features extraction. For example, a corner is easily computed by differentiating the depth of points on the line. Also, the generated 3D data can be encoded as a dense image, as shown in **figure 2a**, which can be then used with various image-based learning algorithms to perform tasks including segmentation and detection. In contrast, the scanning pattern of Livox Horizon is quite irregular, shown in **figure 3**.

- **Non-repetitive scanning:** To maximize the coverage ratio even when the LiDAR is static, non-repetitive scanning is usually adopted where the scanning trajectory never repeats itself. This means that, for solid state sensors, the density of the scanning pattern grows with the integration time; as illustrated in **figure 4a**. The performance of the scanning method is defined by the field-of-view coverage, which is calculated as the fraction of field-of-view illuminated by laser beams. The field-of-view coverage C can be calculated with the following formula:

$$C = \frac{\text{Total area illuminated by laser beams}}{\text{Total area in FOV}} \times 100\% \tag{1}$$

  Figure 4b compares the value of C for different types of sensors.

- **Motion blur:** Due to the continuous scanning of a single laser head, the 3D points measured in one frame are really sampled at different times as the LiDAR is continuously moving. The in-frame motion will distort the point-clouds and cause motion blur. **This effect is severe when the sensor is vibrating**.

3 LiDAR data clustering

This section investigates the feasibility of creating useful clusters from data acquired by Livox Horizon LiDAR. We roughly classify the available methods in the literature into projection-based, and point-cloud-based methods.

3.1 Projection-based methods

Projection-based methods operate on dense image encodings of the LiDAR scans. They provide several benefits over point-cloud-based methods. Firstly, they provide proper data ordering, which makes learning feature extractors more efficient and allows for the utilisation of GPU calculations. Secondly, they allow the usage of

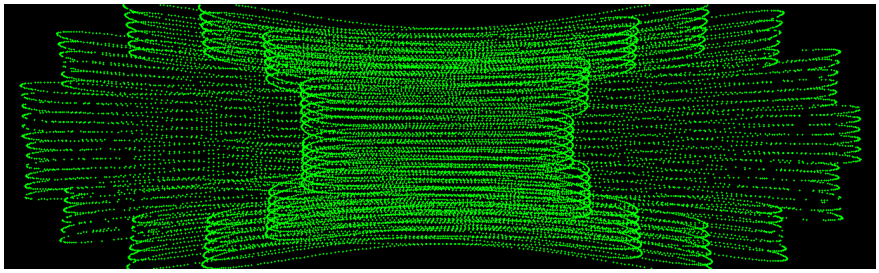
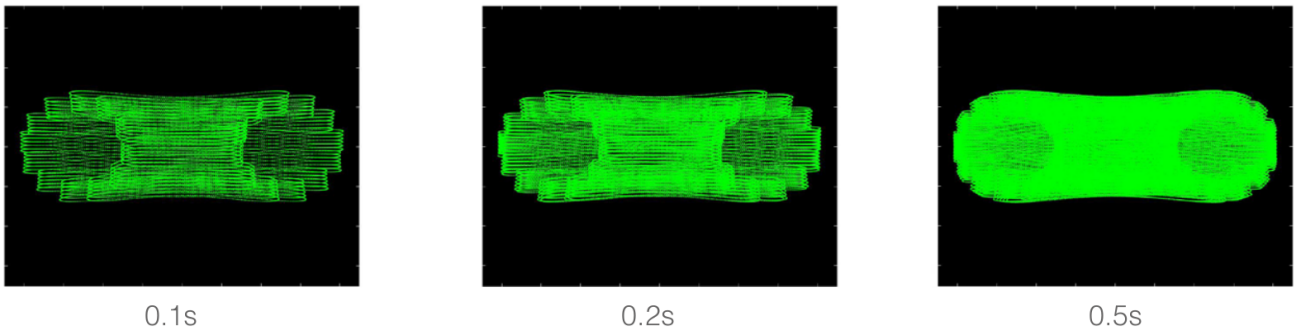
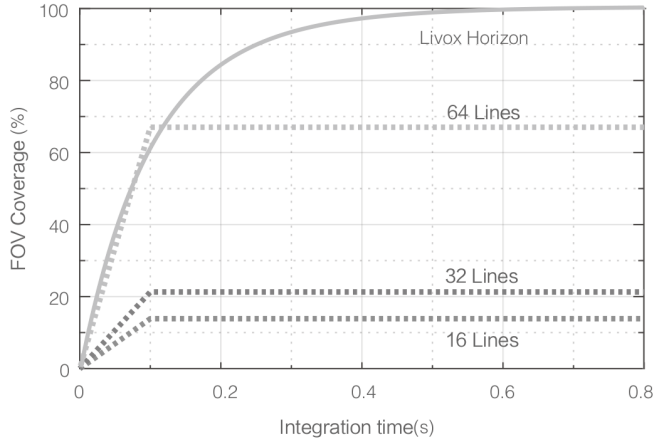


Figure 3: The scanning pattern of the Livox Horizon scanner generated by projecting the 3D scan into a hypothetical image plane at a given field of view. Since the Livox scanner has a narrower field-of-view, an image plane with size 1241×376 encodes the whole scanning pattern. Due to this irregular and non-repeating pattern, encoding data from this LiDAR into a dense depth image is not trivial.

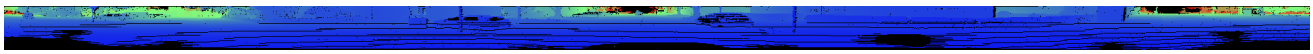


(a) Horizon LiDAR patterns accumulated over extended periods.

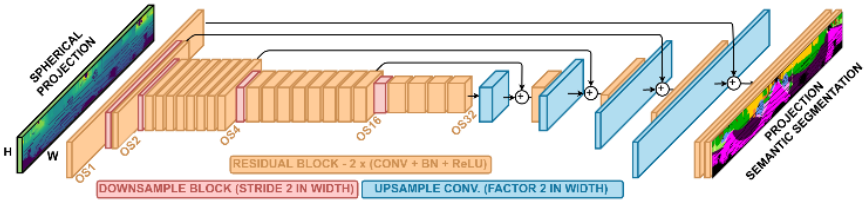


(b) The FOV coverage of the Horizon compared with Velodyne LiDAR sensors that use common mechanical scanning methods. The 16-line Velodyne sensor has a vertical FOV of 30^o, the 32-line Velodyne sensor is 41^o, and the 64-line Velodyne sensor is 27^o. The diagram shows that when the integration time is less than 0.1 seconds, the FOV coverage of the Horizon approaches 60%, similar to the 64-line LiDAR sensor. As the integration time increases to 0.5 seconds, the FOV coverage approaches 100%, so almost all areas are illuminated by laser beams.

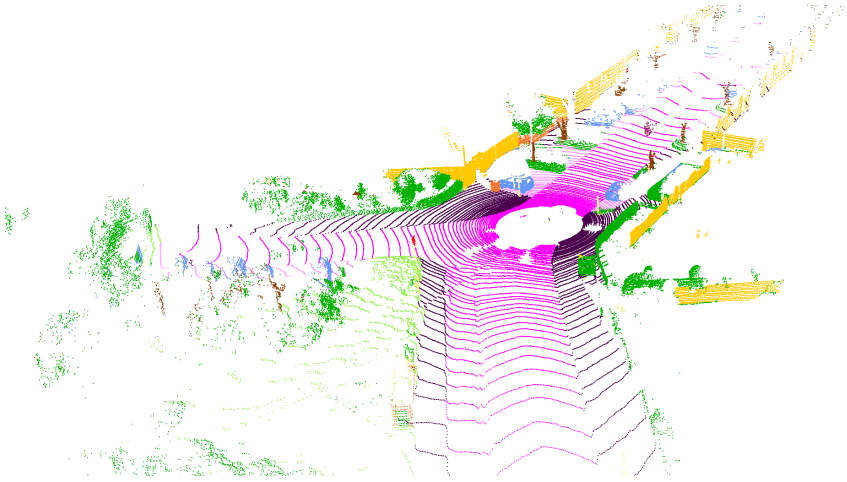
Figure 4: .



(a) LiDAR data is encoded into a dense image projection with a size of $1947 \times 64 \times 5$.



(b) This fully convolutional semantic segmentation architecture with Darknet53 Backbone is used to semantically label objects in the dense image.



(c) Labels are then aggregated to the 3D data.

Figure 5: Semantic segmentation using encoded images in combination with convolutional neural networks.

planar convolutions and overcomes its drawbacks. Thirdly, they make ground points segmentation easier. Finally, they make transferring traditional 2D image-based models to LiDAR data more straight-forward. **Even though, it is not clear how one can encode irregular scanning patterns into dense images, it is of future value to address these methods here and demonstrate their results on regular scanning patterns for comparisons. Thus, the adaptation of these methods to irregular scanning patterns is left for future work.**

One approach to LiDAR data clustering is semantic segmentation using encoded images as an intermediate representation in combination with convolutional neural networks (CNN). This approach infers a class label to each data point in the input data using any CNN as a backbone and exploits GPU-based calculations. This allows us to infer full semantic segmentation of LiDAR point-clouds accurately and faster than the frame rate of the sensor. **Figure 5** shows an example of semantic segmentation of Velodyne HDL64E data using encoder-decoder hour-glass-shaped architecture [1] with darknet backbone [2]. To generate these results, LiDAR data is encoded into a dense 2D image projection with 5-channels (3-channels encoding point location in 3D, 1-channel encoding range, 1-channel encoding return intensity). Training was achieved using publicly available large dataset of semantic annotations [3].

A fast and efficient range image segmentation method that runs at the sensor frame rate is presented in [4]. The key idea of this method is the ability to estimate which measured points originate from the same object for any two neighboring laser beams. To make this decision for a pair of points A and B, the angle β between the laser beam to point A and the line connecting A to B is utilised as a measure. If the change in depth between points A

Figure 6: Depth clustering by thresholding beams angle-of-incident following the method in [4]. The utilised method for ground points removal is rather ad hoc and lacks for robustness.

and B is too large, then β will be too small, and thus we make the decision to separate the points A and B into different segments. With this separation criterion in mind, this segmentation algorithm runs as a breadth-first search in the range image to decide the label of every pixel. These labels are then aggregated into the 3D data. **Figure 6** demonstrates results obtained using this approach on a scene taken with a 64-line Velodyne scanner. One drawback of this geometric approach, and in contrast to semantically segmenting the cloud using CNN, is that it requires ground points to be removed first. Also, utilising a geometric measure, this method tends to over-segment the point-cloud.

3.2 Point-cloud-based methods

Assuming an unorganised set of points, this class of clustering methods operate directly on 3D point-clouds without any intermediate representation. The key idea is to compute a 3D measure upon which the point-cloud is divided into locally coherent clusters. For most of these methods to succeed, initial ground removal is essential. This is also a fundamental step for robust obstacle detection as well. Also, There are typically many more ground points than any other non-ground surface. Thus, ground has enormous redundancy in terms of obtaining accurate results at near real-time.

Given a sensor on a ground vehicle, the following properties might be used to identify ground points:

- The lower LiDAR returns (for a 64-line sensor, the lower 32 lasers, usually, but for Livox Horizon, this has to be tuned) point downwards and typically are completely occupied with ground returns. Thus, the most common surface normal obtained from these returns (e.g., as a histogram) will give a good indication of the dominant ground direction, which might reasonably be taken to define up (in the absence, say, of IMU data).
- The normal vectors of relatively flat ground point approximately upwards. For example, a triangle with 25^o slope has a unit normal that projects onto the vertical axis as 0.9. So, a threshold that accepts projections greater than 0.9 will classify as ground any surfaces with less than 25^o slope.
- Ground points are usually lower than points from other surfaces. Thus, given a nominal set of ground points, as obtained from the surface normals, one might define a ground plane as the average ground height. From this, any points lower than the ground plane might also be classified as ground (even though they may not have given a vertical surface normal).

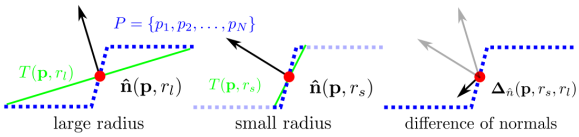


Figure 7: The normal support radius relation to scale.

Detecting and removing ground points following these guidelines is rather **ad hoc** and more complicated in the case of the Livox Horizon due to its irregular pattern. An alternative and more generic way is to utilise the difference of normals computed at the same point with two different scales [5]. This approach is chosen here, because it makes no assumption about the orientation of the sensor with respect to the ground plane or the distribution of the ground points in a scan and does not require ground points to be removed **a-priori**.

The intuition behind the difference of normals (DoN) approach (see **figure 7**) is that if the direction of the two surface normals, estimated with two different support radii (r_s, r_l), is nearly identical, then the structure of the surface does not change significantly from the first radius to the second. By contrast, if the structure of the larger neighborhood around a centre point is significantly different from that of the smaller neighbourhood, then the direction of the two estimated normals are likely to vary by a larger margin. In that case, a value between the two radii is often representative of the scale around near the centre point [5]. This observation can be exploited to segment the point-cloud and remove ground points using the following steps:

- **A multi-scale operator** $\mathbf{L}(\mathbf{p}, r) = \hat{\mathbf{n}}(\mathbf{p}, r)$ for a point-cloud is simply calculated as the normal map $\hat{\mathbf{n}}$ of the point-cloud estimated with support radius r at individual points $\mathbf{p}$. In the most basic case we can compare the response of the operator across two different radii $r_1 < r2$.
- **The DoN operator** $\Delta_{\hat{\mathbf{n}}}$ for any point $\mathbf{p}$ in a point-cloud, is defined as:

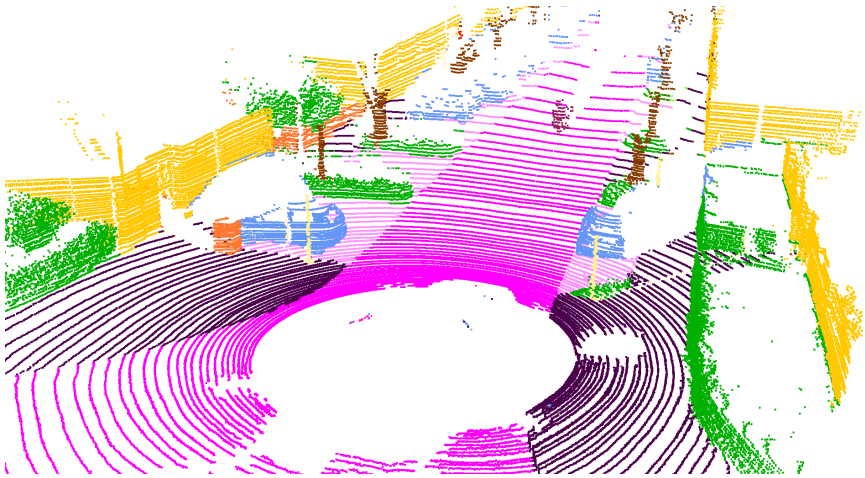
$$\Delta_{\hat{\mathbf{n}}}(\mathbf{p}, r_1, r_2) = \frac{\hat{\mathbf{n}}(\mathbf{p}, r_1) - \hat{\mathbf{n}}(\mathbf{p}, r_2)}{2} \tag{2}$$

  Thus the magnitudes of the DoN vectors are always within $[0, 1]$.

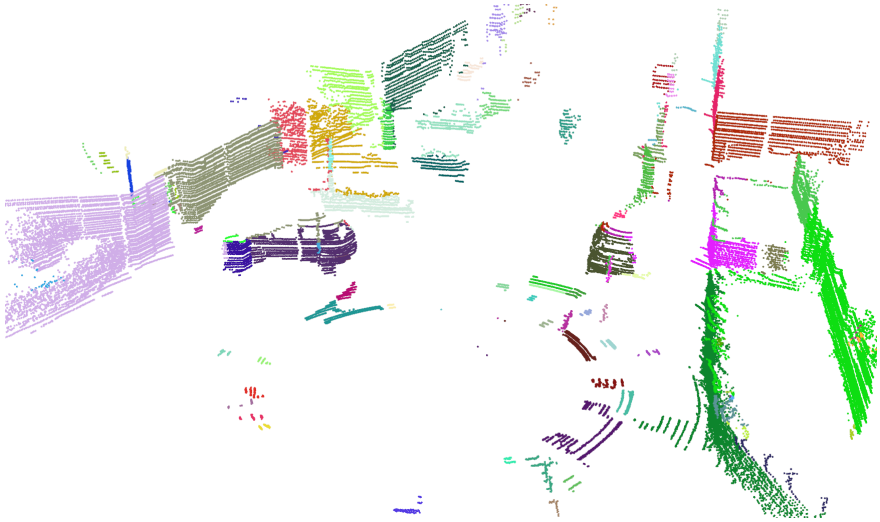
- Estimated DoN magnitudes $||\Delta_{\hat{\mathbf{n}}}(\mathbf{p})||$ are then **filtered** by discarding all points for which $||\Delta_{\hat{\mathbf{n}}}(\mathbf{p})|| < \alpha$. This effectively removes the majority of redundant points from a point-cloud, including ground points.
- A **segmentation** is then obtained using a Euclidean distance threshold based on the following clustering steps,

  1. Create a ks-tree representation for the input point-cloud dataset.
  2. Set up an empty list of clusters C, and a queue of the points that need to be checked Q.
  3. For every point $\mathbf{p}$, perform the following steps:
     - Add $\mathbf{p}$ to the current queue Q.
     - For every point $\mathbf{p} \in Q$, search for the set of point neighbours of $\mathbf{p}$ in a sphere with radius β. For every neighbor, check if the point has already been processed, and if not add it to Q.
     - When the list of all points in Q has been processed, add Q to the list of clusters C, and reset Q to an empty list.
  4. Terminate when all points have been processed and are now part of the list of point clusters C.

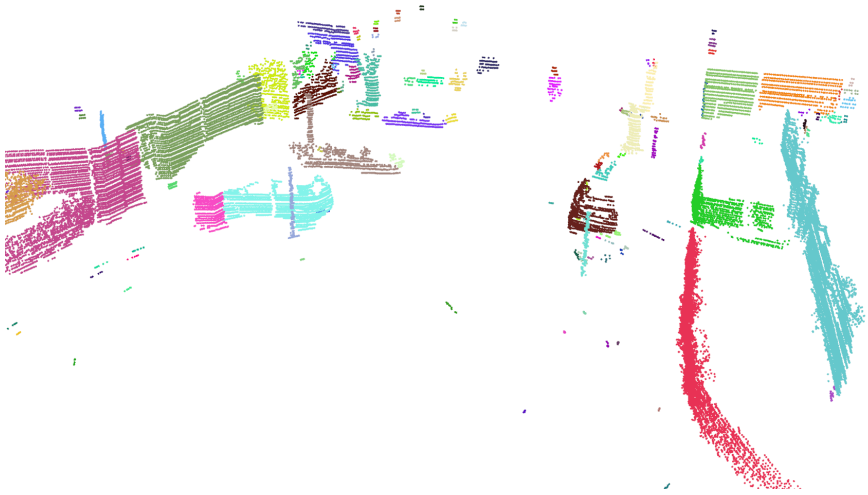
Figure 8 compares results from the three discussed methods applied to data from a 64-line sensor, while **figures 9 to 14** show the application of Livox Horizon point-cloud segmentation using the DoN operator as a measure having a distance tolerance $\beta = r_1$, a minimum of 100 cluster points, and a maximum of 100,000 cluster points, for different types of obstacles. **Figure 12** shows the effect of vibrations on filtering and segmentation outcome. Finally, **figure 15** shows the effect of changing the radii (r_1, r_2) on DoN filtering results.



(a) Semantic segmentation using CNN with Darknet53 backbone.

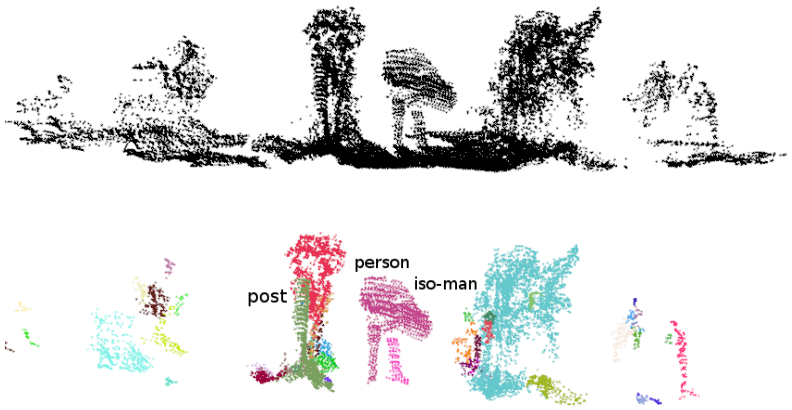


(b) Depth clustering using angular difference.

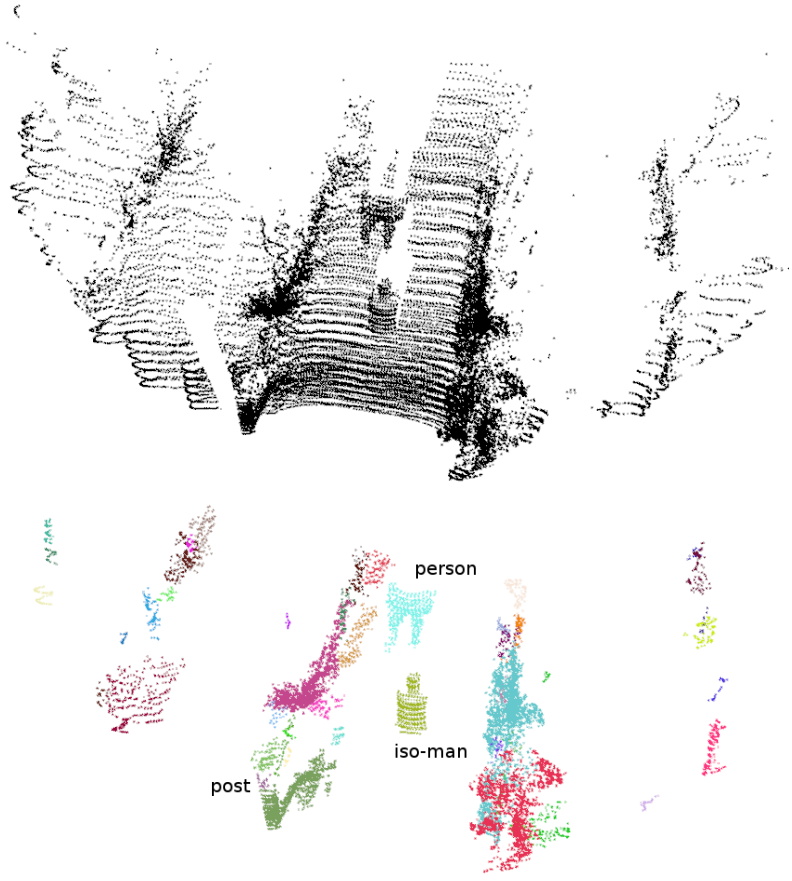


(c) Clustering using difference of normals.

Figure 8: A comparison of clustering methods on an urban scene scanned using a 64-line sensor. Unlike semantic segmentation using CNN in **8a**, both geometric methods used to create **8b** and **8c** returned over-segmented clusters.

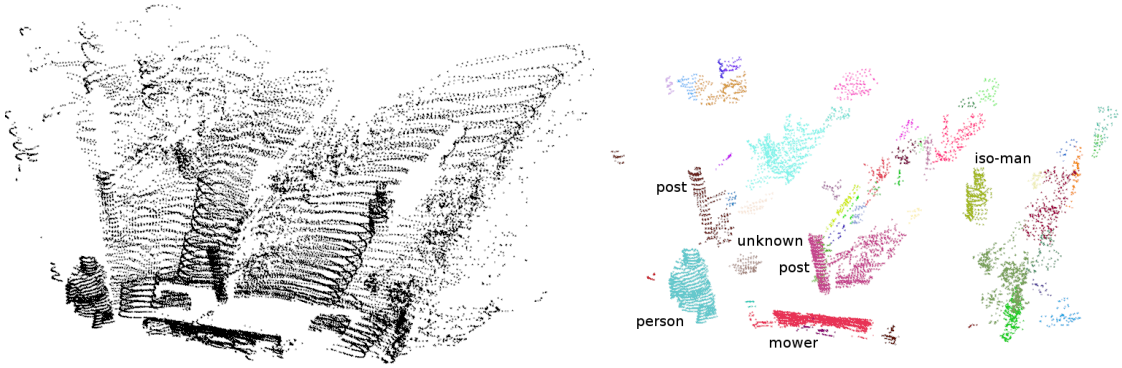


(a)

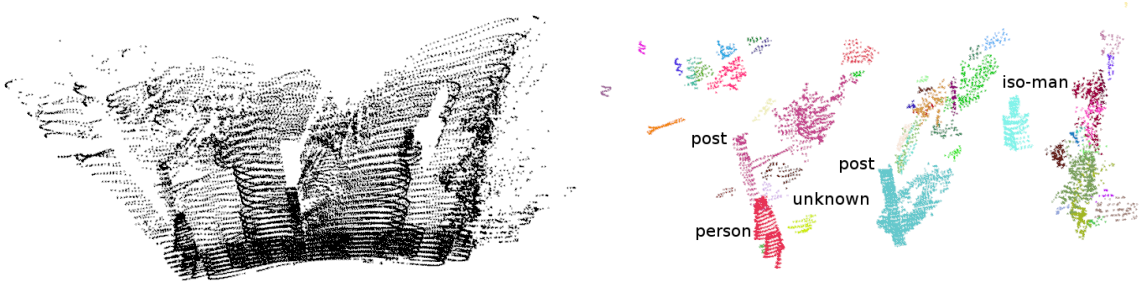


(b)

Figure 9: Clustering on Livox Horizon data using DoN operator.



(a)



(b)

Figure 10: Clustering on Livox Horizon data using the DoN operator.

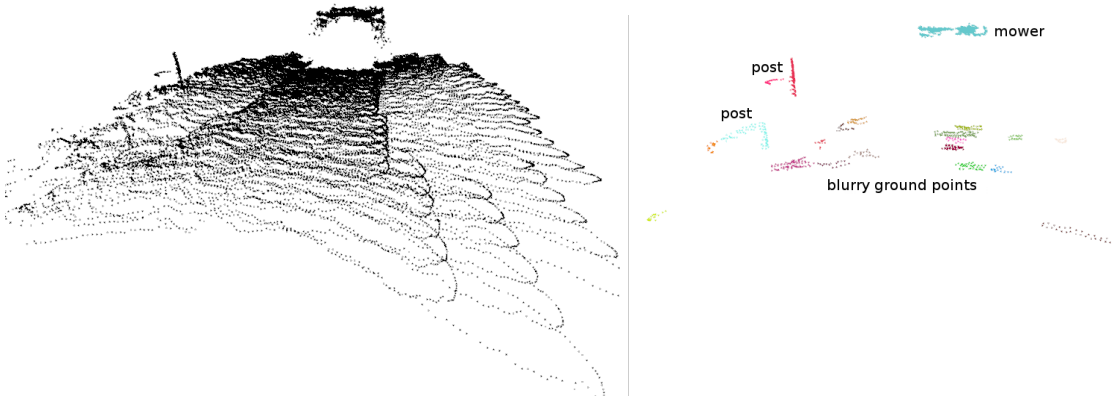


Figure 11: The effect of vibrations: the figures demonstrate how, when vibrations are present, ground points altitude may vary creating blurry point-clouds. This results in some ground points persisting in the segmented cloud.

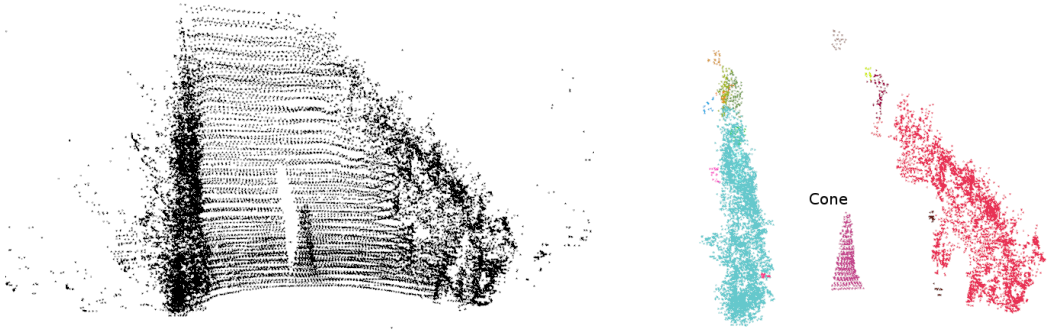


Figure 12: Clustering on Livox Horizon data using the DoN operator.

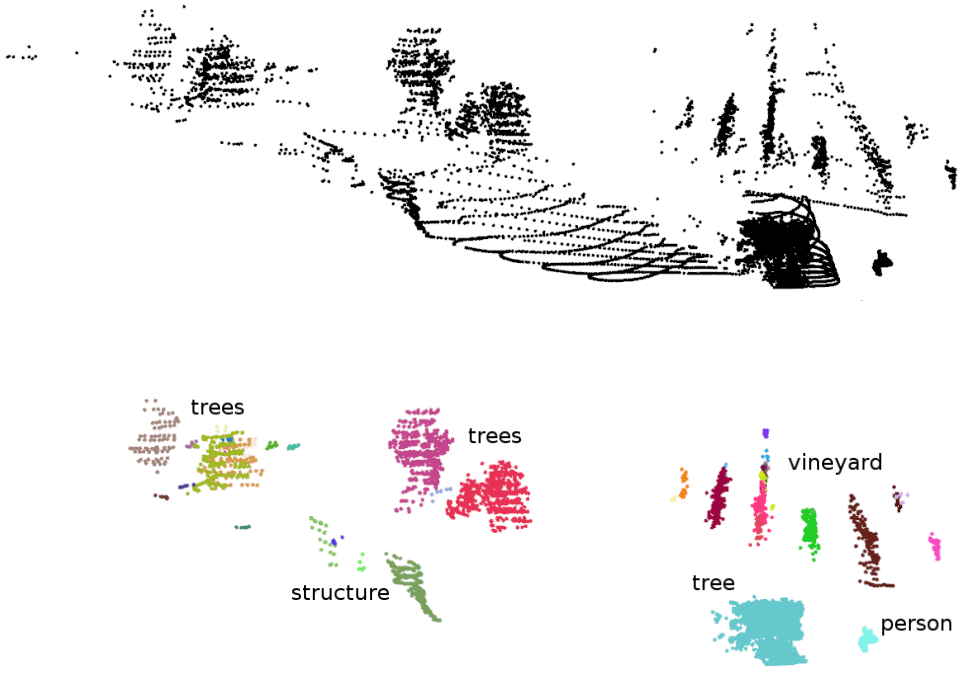
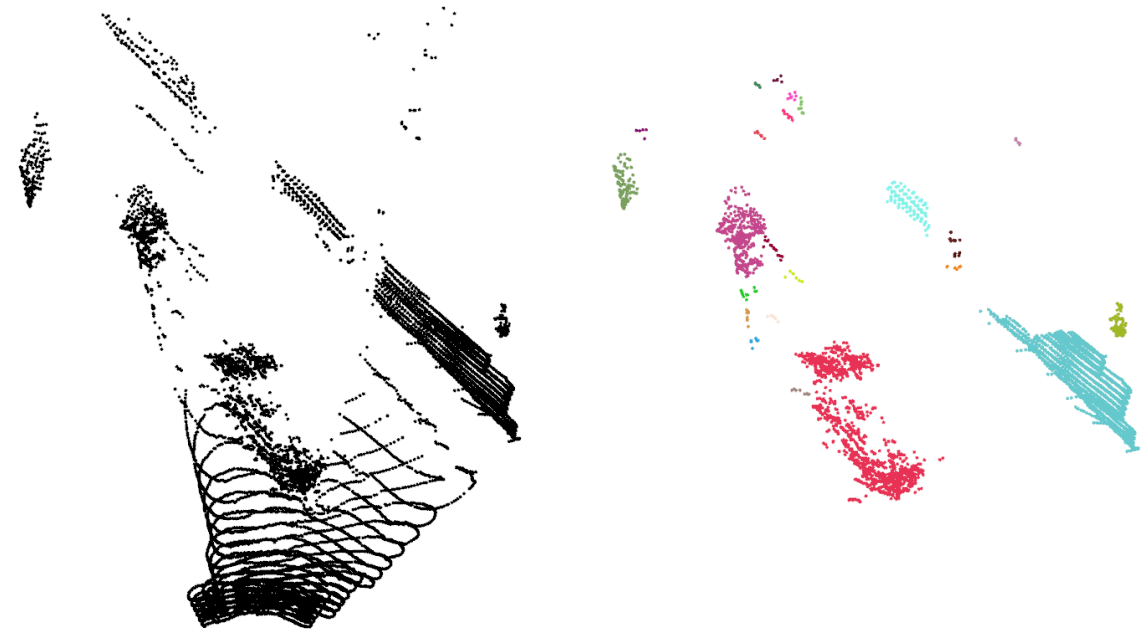
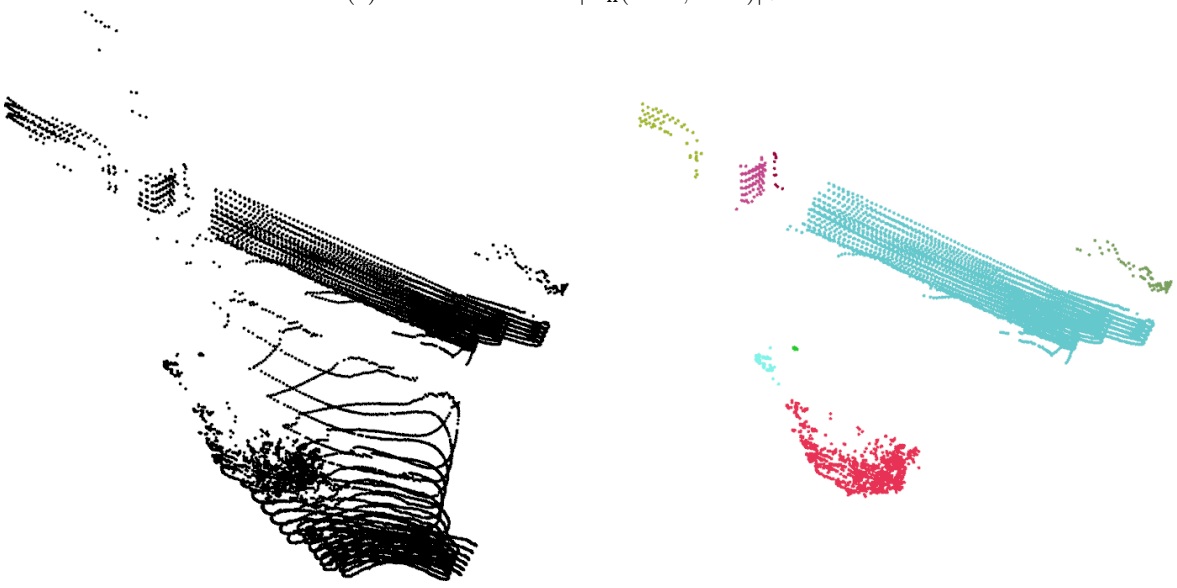


Figure 13: Clustering on Livox data using the DoN operator at the proximity of the vineyard.

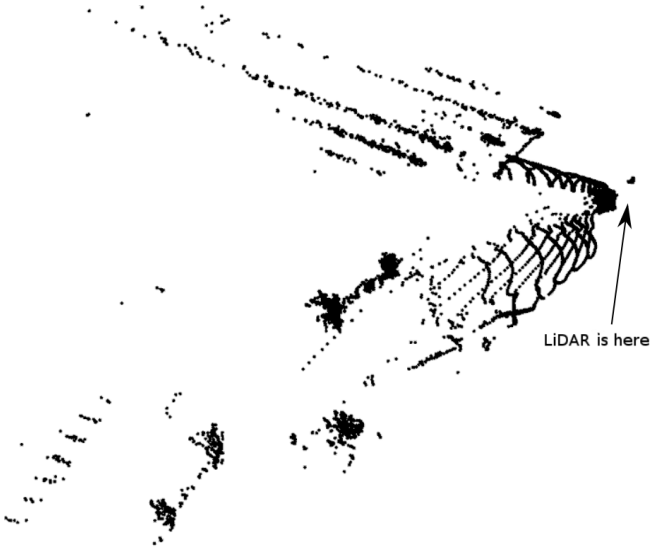


(a) Clusters found in $|\Delta_{\hat{\mathbf{n}}}(0.8m, 8.0m)| > 0.25$.



(b) Clusters found in $|\Delta_{\hat{\mathbf{n}}}(0.8m, 8.0m)| > 0.25$.

Figure 14: This figure illustrates the effect of distance to objects and their geometry on DoN filtering results. In figure 14b the sensor is closer to the wall than in figure 14a. The figure also shows the irregular and non-repetitive nature of the scanning pattern (the footprints of the two scans don't match).



(a) scan.

(b) Clusters found in $|\Delta_{\hat{\mathbf{n}}}(0.2m, 2.0m)| > 0.25$.

(c) Clusters found in $|\Delta_{\hat{\mathbf{n}}}(0.4m, 4.0m)| > 0.25$.

(d) Clusters found in $|\Delta_{\hat{\mathbf{n}}}(0.8m, 8.0m)| > 0.25$.

Figure 15: DoN filtering results for various radii. These scans are viewed from the top. Figure 15a depicts the raw scan points and the remaining figures show the result of DoN filtering applied to the raw scan. As the scale is increased, larger and larger objects are preserved.

4 Localisation and mapping

The focus of this section is to investigate the feasibility of building consistent and scalable 3D maps using the Livox Horizon LiDAR **in real-time**. The application value of this may include:

- **Mapping vineyard fields:** Fruit orchards mapping services may include estimation of canopy volume, estimation of crop yield, monitoring its layout, and change detection due to environmental and weather changes. Most of these services benefit from a large scale and highly detailed orchard map.
- **Robust obstacle detection:** Decisions made on free-space and non-free-space become incrementally more robust and accurate as more scans are incorporated into the **local map**.
- **Navigation in GPS-denied areas:** Most of available autonomous robots working in the field utilise GPS-base localisation strategies. They can achieve high accuracy solutions at good satellite constellation. Yet, they are prone to errors due to GPU-signal blockage by canopy and multi-path from nearby structures. Thus, an autonomous robot could benefit from more sensor modalities including LiDARs and vision.

LiDAR SLAM algorithms have witnessed significant improvements in accuracy, scalability and efficiency [6], [7]. They benefit from regular and correspondence preserving scanning patterns of rotary scanners in creating efficient scan matching results that can run in real-time (reported 10Hz and 20Hz output) and can scale to large outdoor environments. However, resources on utilising solid-state LiDARs are limited, because they are relatively new. Nevertheless, there is a significant similarity between these sensors and traditional small field-of-view sensors. The only notable differences are in the utilised features matching algorithms [8]. To overcome the irregularity problem, an efficient and scalable database is needed to store and retrieve 4D features (x, y, z-point and intensity) at frame rate. One example is k-d tree [9], which is a binary search tree in which every leaf node represents a k-dimensional feature point. K-d tree has $\mathcal{O}(n)$ worst space complexity, and $\mathcal{O}(\log\, n)$ average time complexity for tasks including search, insert and delete. Also, Livox Horizon, has an internal IMU, where inertial pre-integration can be used as an initialisation point for scans matching. thereby eliminating ambiguities and improving localisation efficiency due to the small field of view.

Figure 16 shows scans accumulated within a window while traveling in a straight line and then while turning in **figure 17**. These experiments highlight the negative impact the narrow field of view of the Livox Horizon has on scans alignment and hence vehicle localisation. The figures also show that this issue can be mitigated by incorporating an IMU to estimate a more accurate initial alignment between the scans. This is then further refined using ICP on the extracted set of features. These experiments were conducted using scans with field-of-view shown on the left side of **figure 18a** and running at 20Hz.

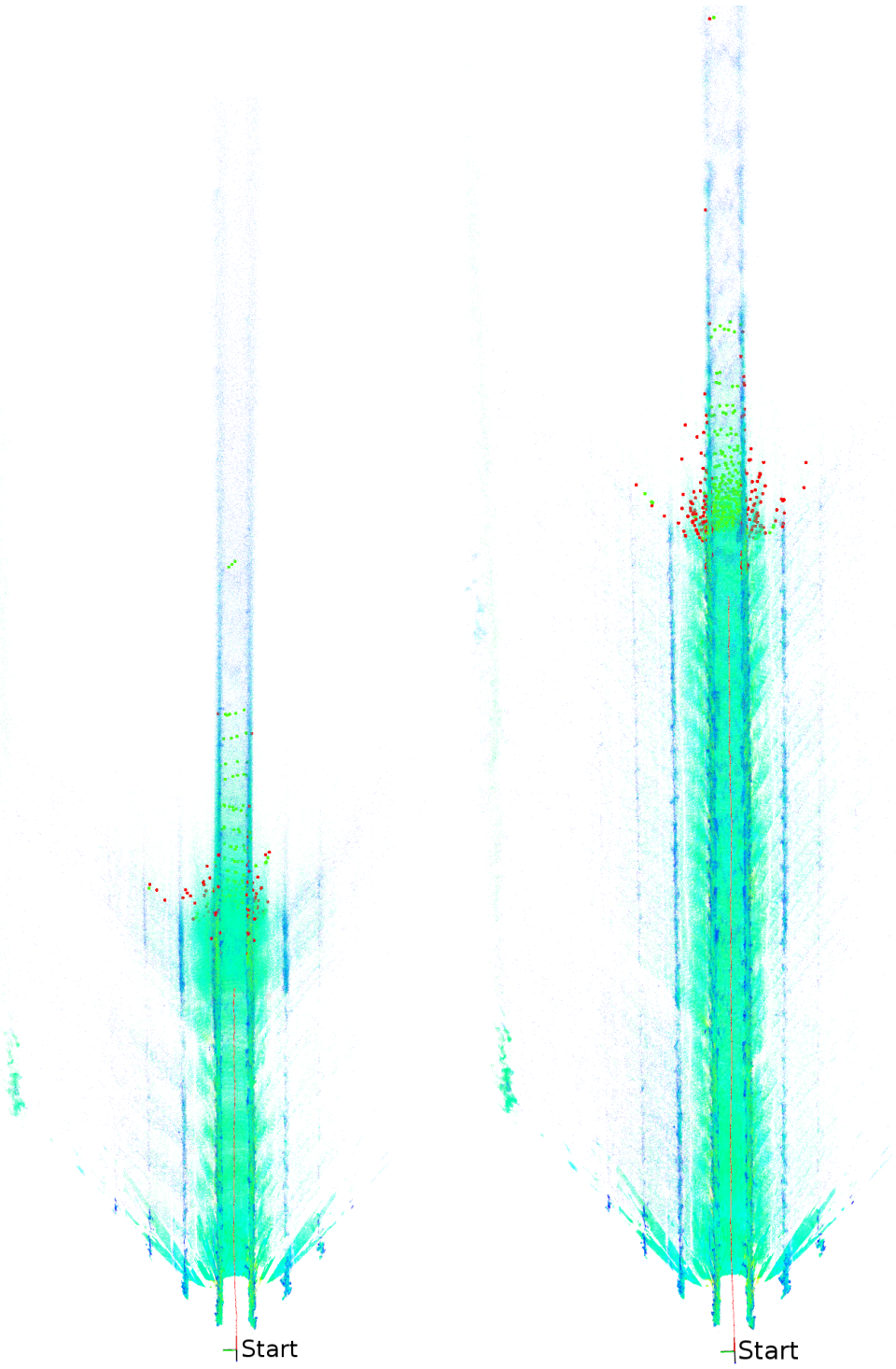


Figure 16: LiDAR scans accumulated over 60 seconds. Top is using LiDAR odometry only, while bottom is using lidar and IMU odometry. The figure shows that the travel distance of the vehicle was under estimated without an IMU. Hence, the scans were also mis-registered. The red markers at the end of trajectory represent the features used during scans alignment, while the vehicle trajectory is plotted in red.

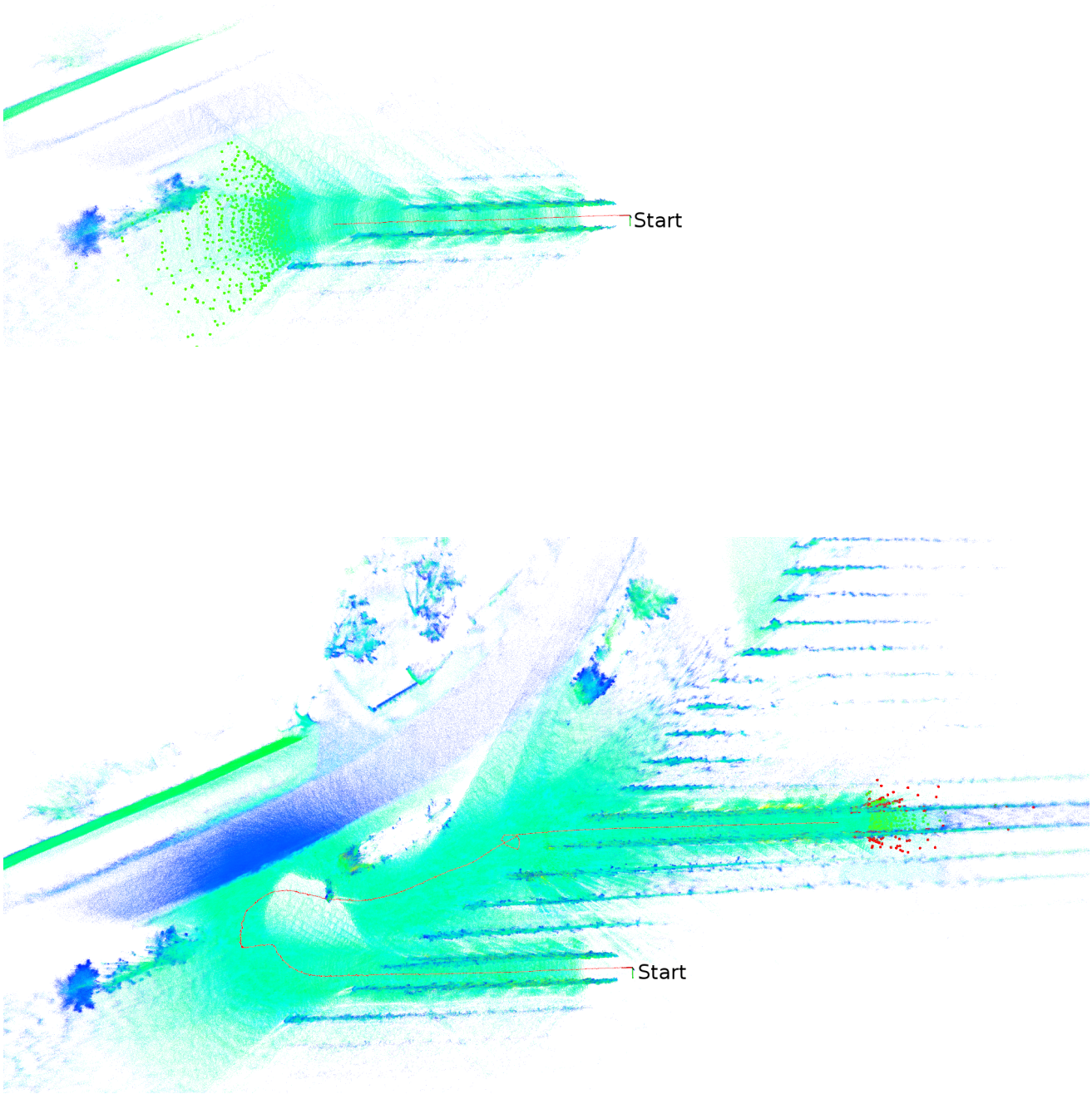


Figure 17: LiDAR scans accumulated over 130 seconds, during which the vehicle was at the end of a row and turning into another one. Top is using LiDAR odometry only, while bottom is using LiDAR and IMU odometry. The figure shows that LiDAR-only odometry failed due to lost data associations between the scans. The red markers at the end of trajectory represent the features used during scans alignment, while the vehicle trajectory is plotted in red.

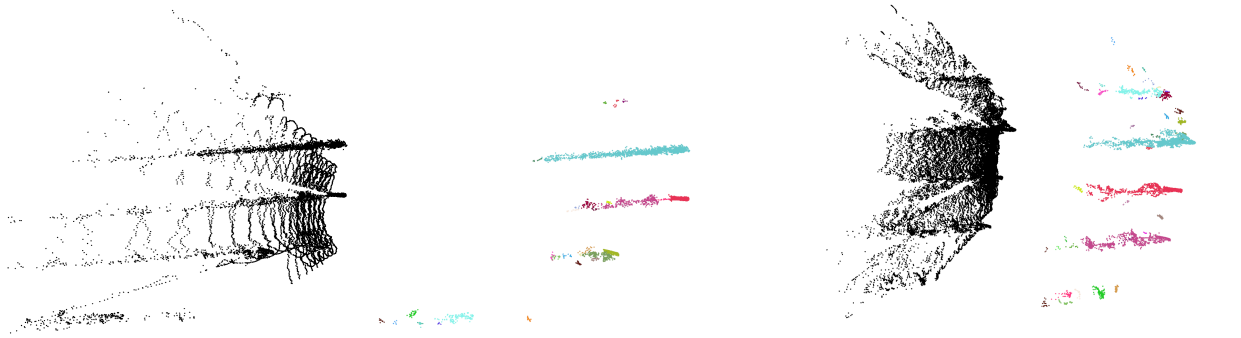
5 Developed software

During this study, the following tools were developed using C++ to process the scans:

- Data conversion tools: the following tools were utilised to inspect the data and generate formats compatible with existing source.
  - **lvx2bag**: converts an lvx file to rosbag format. A rosbag can then be used to generate the 3D map and replay the canning process.
  - **bag2pcd**: converts a rosbag to a pcd format. This is an intermediate format that generate a single pcd file for each scan.
  - **pcd2bin**: converts a pcd file to bin file. This format is compatible with the Kitti dataset, and hence can be used for training and inference.
  - **bin2pattern**: converts a bin file to a png image containing the scanning pattern. The pattern image can be used to inspect and visualise the scan.
  - **bin2png**: converts a bin file to a png image containing the canning pattern with depth information.
  - **bin2depth**: converts a bin file to a png image containing dense depth data, which can be then used to infer segmentations using projection-based methods. Currently, this works only for the HDL64E scanner.
- Point-cloud-based segmentation and visualisation tools:
  - **DoN**: a class containing all methods required to estimate DoN and perform point-cloud segmentation using DoN operator.
  - **CloudViewer**: a class containing all methods required for 3D visualisation of point-cloud segmentations. This implementation is based on Pangolin [1], which manages OpenGL display and interaction.
  - **example_don**: a sample script to perform segmentation on a single scan. This only accepts bin files at the moment.
  - **example_don_gpu**: a sample script to perform segmentation using DoN operator on a GPU. This has not been tested yet due to GPU memory requirements.
  - **show_separated_cluster_files**: a function that takes in .bin cluster files and returns a color coded map of clusters.

All the examples of projection-based methods presented in this report were implemented during previous projects using data from mechanically spinning LiDARs and utilised publicly available libraries [2] and datasets [3].

[1] https://github.com/stevenlovegrove/Pangolin
[2] https://github.com/PRBonn/semantic-kitti-api
[3] http://semantic-kitti.org/



(a) From lower position and at smaller pitch angle (12,000 points at 20Hz).

(b) From higher position and at larger pitch angle (24,000 points at 20Hz).

Figure 18: The effect of integration time and sensor placement is evident. Filtering and segmentation using DoN operator benefit from ground points in estimating which points belong to meaningful clusters. As the sparsity of the points increases with the observed range, more points are being removed due to insufficient supporting scale. Thus, at the given frequencies, both setups have a comparable obstacle detection range when using DoN filtering. It is expected that if the sampling rate remains fixed (at 10Hz), we should observe a slight increase in detection range of the case 18a.

6 Conclusions

We have analysed the LiDAR data by the Livox Horizon, and concluded that **the scanning pattern of this sensor limits the number of current methods that we can use** to detect potential obstacles in the way of the vehicle. Also, we have shown that one could apply heuristics to label the ground points without any guarantees of consistency or robustness of the results.

At this stage, we have found that **operating directly on the 3D point-cloud is more suitable for this type of sensors**, despite the facts that it takes longer time to process, it requires more memory to store, and it tends to over-segment the point-cloud. Even-though we have limited our study to geometric 3D methods, such as DoN, there exist methods that operate on 3D point-clouds using machine learning or deep learning, but the application of these methods to data from Livox Horizon could be limited by the fact that the scanning process is different from the ones used to generate publicly available annotations. **It is, however, worthwhile to**:

- **Investigate** the possibility of **dense image encoding** of the Horizon data to benefit from existing projection-based methods.
- **Investigate** learning how-to-segment 3D point-clouds using **deep neural networks**.

Figure 18 shows data collected from two different sensor mounting locations and tilt angles. Here, we consider the effect of sensor placement on the performance of segmentation using DoN, while we are unable to make any decision on its effect on the vehicle localisation and mapping at this stage.

Initial investigation of the feasibility of scans matching and alignment, and vehicle pose estimation shows that **a navigation solution is possible at real-time. The only exception is loop closure**, which we are unable to make any decisions due to insufficient data.

7 Moving forward

- **Real-time DoN**: As suggested in this work, DoN operator can be a powerfull tool to remove non-obstacle data points from scans, and generate rough segmentation. However, multi-scale normal calculations on CPU take long time, especially at large scales and higher point densities. For our experiments, it took 4 seconds to compute DoN using two scales, $r_1 = 0.8m$ and $r_2 = 8.0m$, using 24,000 points. Calculating the two normal maps estimated with support radii r_1 and r_2 for a scene is a process which is highly parallelizable and thus greatly benefits from GPU optimization.

- **Dense image encoding of LiDAR scans**: Inference using CNN can be made cheap, for instance using TensorRT, on embedded systems. It is interesting to investigate how one can deploy projection-based methods for data with irregular patterns. There are two sub-directions here, the first, investigates if one can generate dense image encodings similar to those of 64-line scanners and then use publicly available resources for training and inference. The second direction, assumes that the two image encodings are different, and hence finds the suitable one, and then builds it database of annotations for training.

- **LiDAR and vision**: LiDAR data segmentation relies solely on geometry, learnt or estimated. For point-clouds segmentation using DoN operator, it is difficult to find optimal set of filtering and clustering parameters (α and β, respectively). A strict filtering strategy (larger α) will result into some obstacle being removed, for instance rough terrain, and vice versa, loose filtering strategy (smaller α) will result into many false alarm being raised. Another complicated situation is vegetation, where grass could be considered an obstacle. Making use of visual object recognition allows for more reliable point cloud labeling.

- **LiDAR loop closure**: When traveling, localisation errors are largest in the horizontal plane (x and y directions). They are usually corrected during scan alignment step. However, errors in the vertical z direction are usually harder to observe and small increments will result in a drift building up over time. This drift is more explicit when the sensor observes a previously visited features, and hence can correct this drift in all directions. One last step needed here is to investigate if loop closure using Livox Horizon is feasible.

References

[1] A. Milioto, I. Vizzo, J. Behley and C. Stachniss, "RangeNet++: Fast and Accurate LiDAR Semantic Segmentation," 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Macau, China, 2019, pp. 4213-4220, doi: 10.1109/IROS40897.2019.8967762.

[2] Redmon, J. & Farhadi, A. (2018), 'YOLOv3: An Incremental Improvement', cite arxiv:1804.02767Comment: Tech Report.

[3] J. Behley, M. Garbade, A. Milioto, J. Quenzel, S. Behnke, C. Stachniss and J. Gall. (2019), "SemanticKITTI: A Dataset for Semantic Scene Understanding of LiDAR Sequences", Proc. of the IEEE/CVF International Conf. on Computer Vision (ICCV).

[4] I. Bogoslavskyi and C. Stachniss (2016), "Fast Range Image-Based Segmentation of Sparse 3D Laser Scans for Online Operation", Proc. of The International Conference on Intelligent Robots and Systems (IROS).

[5] Y. Ioannou, B. Taati, R. Harrap and M. Greenspan, "Difference of Normals as a Multi-scale Operator in Unorganized point-clouds," 2012 Second International Conference on 3D Imaging, Modeling, Processing, Visualization & Transmission, Zurich, 2012, pp. 501-508, doi: 10.1109/3DIMPVT.2012.12.

[6] Shan, Tixiao and Englot, Brendan, "LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain", IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2018.

[7] https://github.com/HKUST-Aerial-Robotics/A-LOAM

[8] Lin, Jiarong and Fu Zhang. Loam_livox: A fast, robust, high-precision LiDAR odometry and mapping package for LiDARs of small FoV. ArXiv abs/1909.06700 (2019).

[9] https://en.wikipedia.org/wiki/K-d_tree